

MYSQL ASSIGNMENT 2-Querying Data

Project Objective:

The focus of this project is on Data Insertion, querying, filtering, aggregation and result interpretation to ensure consistency with existing database design. The Employee Database already contains required tables, relationships and Constraints.

Dataset Description:

The given Dataset is the Employee Database which contains Three tables such as employees, departments and locations of the [organisation](#). For each table we have created the attributes with specific constraints.

Column Description:

Table:Department

Column Name	Data Type
Department ID	It should be unique so it is Primary Key
Department Name	It contains Department Name, it is VARCHAR

Table:Location

Column Name	Data Type
Location ID	It should be unique so it is primary key
Location Name	It contains Location, it is VARCHAR data type

Table:Employees

Column Name	Data Type
Employee ID	It is primary key because it should be unique.
Employee Name	It contains name of the employee so it is VARCHAR data type
Gender	It should be either male or female,it is ENUM data type
Age	Age of the employee should be INT
Hire Date	Hiring date is in DATA data type
Salary	It is in decimal data type
Designation	It is in VARCHAR data type

In the employees table,Department ID and Location ID are Foreign Key because they are references from Department and location tables.

DML Insertion:

Employee Database is created in last assignment to populate the table input values are given using INSERT INTO command to three tables.

Command Used:

```
INSERT INTO department (Department_id,Department_name)
```

```
VALUES
```

```
(101,'Human Resource'),
```

```
(102,'Marketing'),
```

```
(103,'Finance'),
```

```
(104,'Sales'),  
(105,'IT'),  
(106,'Administration'),  
(107,'Customer Service'),  
(108,'Operations');  
SELECT*FROM department;
```

```
INSERT INTO location(Location_id,Location_name)  
VALUES(201,'Chennai'),  
(202,'Mumbai'),  
(203,'Delhi'),  
(204,'Kolkatta'),  
(205,'Uttar Pradesh'),  
(206,'Bangalore'),  
(207,'Maharashtra'),  
(208,'Hyderabad');  
SELECT*FROM location;
```

```
INSERT INTO  
employees(Employee_id,Employee_Name,Gender,Age,Hire_Date,Designa  
tion,Department_id,Location_id,Salary)  
VALUES(1001,'Ram','M',23,'2026-06-05','Data Analyst',101,201,55000),  
(1002,'Janani','F',31,'2015-08-01','Manager',102,202,62000),  
(1003,'Nivas','M',24,'2016-04-11',' ',103,203,15000),  
(1004,'Priya','F',33,'2018-02-17','HR Executive',104,204,35000),
```

(1005,'Seetha','F',47,'2015-12-09','IT Manager',105,205,43000),
 (1006,'Ravi','M',51,'2017-08-26','Marketing Executive',106,206,24000),
 (1007,'Kavitha','F',39,'2019-01-27','Administrative Assistant',107,207,31000),
 (1008,'Sathish','M',21,'2026-03-19','Customer Service Executive',108,208,17000);

Clause&operators:

1.A query to retrieve distinct salaries from employee table:

Command Used:SELECT*FROM employees;
 SELECT DISTINCT Salary FROM employees;

Result Grid	
	Salary
▶	18000.00
	40000.00
	15000.00
	35000.00
	43000.00
	24000.00
	31000.00
	17000.00

2.Provide Alias for 'Age ' and 'Salary column:

Command Used:SELECT AGE AS Employee_Age,Salary AS Employee_Salary FROM employees;

Result Grid   Filter Rows:

Employee_Age	Employee_Salary
23	55000.00
31	62000.00
24	15000.00
33	35000.00
47	43000.00
51	24000.00
39	31000.00
21	17000.00

3.Retrieve employees with salary greater than 50000 and hired before 2016-01-01.

Command Used:SELECT*FROM employees WHERE Salary>50000 AND Hire_Date< '2016-01-01';

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Department_id	Location_id	Salary	Email
▶	1002	Janani	F	31	2015-08-01	Manager	102	202	62000.00	NULL
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4.Find employee whose designation is missing and fill it with 'Data Scientist':

Command Used:UPDATE employees

SET Designation=null

WHERE Employee_id=1003;

SELECT*FROM employees;

UPDATE employees

SET Designation='Data Scientist'

WHERE Employee_id=1003;

SELECT*FROM employees;

Result Grid										
Filter Rows:										
Edit: Export/Import: Wrap Cell Content:										
Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Department_id	Location_id	Salary	Email	
1001	Ram	M	23	2026-06-05	Data Analyst	101	201	55000.00	NULL	
1002	Janani	F	31	2015-08-01	Manager	102	202	62000.00	NULL	
1003	Nivas	M	24	2016-04-11	Data Scientist	103	203	15000.00	NULL	
1004	Priya	F	33	2018-02-17	HR Executive	104	204	35000.00	NULL	
1005	Seetha	F	47	2015-12-09	IT Manager	105	205	43000.00	NULL	
1006	Ravi	M	51	2017-08-26	Marketing Executive	106	206	24000.00	NULL	
1007	Kavitha	F	39	2019-01-27	Administrative Assistant	107	207	31000.00	NULL	
1008	Sathish	M	21	2026-03-19	Customer Service Executive	108	208	17000.00	NULL	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Employee ID 1003 has null designation which is replaced with 'Data Scientist' designation.

Sorting and Grouping Data:

1.Sort employees by department ID in ascending order and salary by descending order:

Command Used:SELECT Department_id,Salary

FROM employees

ORDER BY Department_id ASC,Salary DESC;

Department_id	Salary
101	55000.00
102	62000.00
103	15000.00
104	35000.00
105	43000.00
106	24000.00
107	31000.00
108	17000.00

2.Display first 5 employees hired in year 2018:

Command Used:SELECT * FROM employees

WHERE YEAR(Hire_Date)=2018

ORDER BY Hire_date DESC

LIMIT 5;

Result Grid										Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Department_id	Location_id	Salary	Email							
	1004	Priya	F	33	2018-02-17	HR Executive	104	204	35000.00	NULL							
	NULL	NULL	NULL	NULL	NULL	HR Executive	NULL	NULL	NULL	NULL							

3. Calculate sum of all salaries in Finance Department:

Command Used: SELECT SUM(e.salary) AS 'Total Salary', d.Department_Name

FROM employees AS e

INNER JOIN department AS d ON e.Department_id=d.Department_id

WHERE d.Department_Name='Finamnce' ;

Using Inner Join salaries are calculated for finance department.

Result Grid		Filter Rows:	Exp
	Total Salary	Department_Name	
▶	15000.00	Finamnce	

4. Find Minimum age among all employees:

Command Used: SELECT MIN(Age) AS 'Minimum age' FROM employees;

Minimum age
21

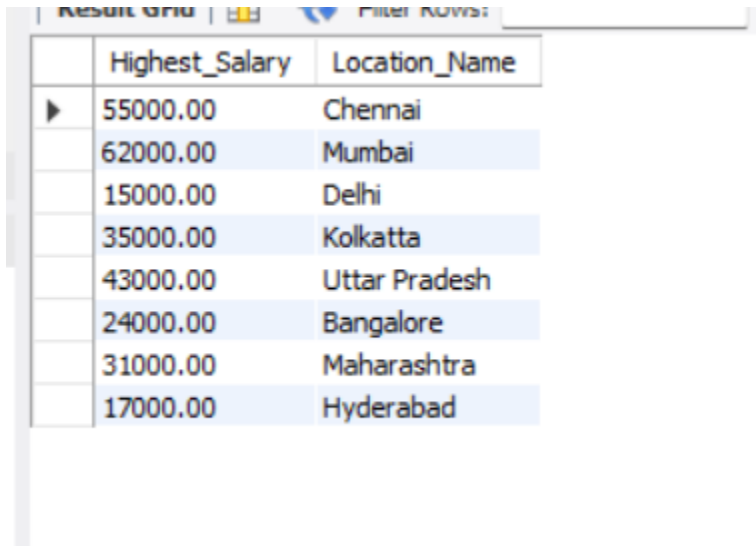
5. List Maximum Salary for each Location:

Command Used: SELECT MAX(e.Salary) AS 'Highest_Salary', l.Location_Name

FROM employees AS e

INNER JOIN location AS l ON e.Location_id=l.Location_id

GROUP BY Location_Name;



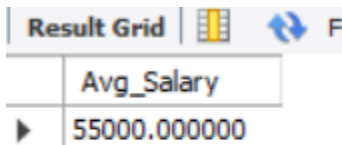
The screenshot shows a 'Result Grid' window with a table containing two columns: 'Highest_Salary' and 'Location_Name'. The table lists the highest salary for each of the following locations: Chennai (55000.00), Mumbai (62000.00), Delhi (15000.00), Kolkatta (35000.00), Uttar Pradesh (43000.00), Bangalore (24000.00), Maharashtra (31000.00), and Hyderabad (17000.00).

Highest_Salary	Location_Name
55000.00	Chennai
62000.00	Mumbai
15000.00	Delhi
35000.00	Kolkatta
43000.00	Uttar Pradesh
24000.00	Bangalore
31000.00	Maharashtra
17000.00	Hyderabad

Inner join is used here to calculate salary for each location.

6.Average Salary for each designation containing word 'analyst':

Command Used: SELECT AVG(Salary) AS Avg_Salary FROM employees
WHERE Designation LIKE '%Analyst' ;



The screenshot shows a 'Result Grid' window with a table containing one column: 'Avg_Salary'. The table lists the average salary for designations containing the word 'analyst', which is 55000.000000.

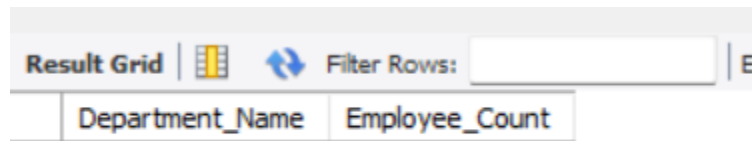
Avg_Salary
55000.000000

7.Department with less than 3 employees:

Command Used: SELECT d.Department_Name,
COUNT(e.Employee_id) AS Employee_Count
FROM department AS d
INNER JOIN employees AS e
ON d.Department_id=e.Department_id

GROUP BY d.Department_Name

HAVING COUNT(e.Employee_id)>3;



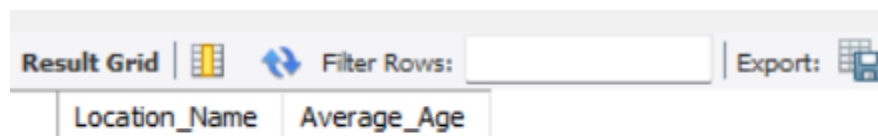
The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', a grid icon, a refresh icon, 'Filter Rows:', and an 'Export' button. Below the toolbar is a table with two columns: 'Department_Name' and 'Employee_Count'.

Department_Name	Employee_Count
-----------------	----------------

8.Location with female employees whose average age is below 30:

Command Used:

```
SELECT l.Location_Name,  
AVG(e.Age) AS Average_Age  
FROM employees AS e  
INNER JOIN location AS l  
ON e.Location_id=l.Location_id  
WHERE e.Gender='F'  
GROUP BY l.Location_Name  
HAVING AVG(e.Age)<30;
```



The screenshot shows a database interface with a toolbar at the top containing 'Result Grid', a grid icon, a refresh icon, 'Filter Rows:', and an 'Export' button. Below the toolbar is a table with two columns: 'Location_Name' and 'Average_Age'.

Location_Name	Average_Age
---------------	-------------

Joins:

1.Inner Join:

List employee names, designation and department names where employees are assigned to a department

Command Used:

```
SELECT e.Employee_Name,e.Designation,d.Department_Name
FROM employees AS e
INNER JOIN department AS d
ON e.Department_id=d.Department_id;
```



The screenshot shows a database query result grid with the following data:

Employee_Name	Designation	Department_Name
Ram	Data Analyst	Human Resource
Janani	Manager	Marketing
Nivas	Data Scientist	Finamnce
Priya	HR Executive	Sales
Seetha	IT Manager	IT
Ravi	Marketing Executive	Administration
Kavitha	Administrative Assistant	Customer Service
Sathish	Customer Service Executive	Operations

2. Left Join:

List all departments with total number of employees in each department including with no employees.

Command Used: SELECT d.Department_Name,

COUNT(e.Employee_id) AS Total_Employees

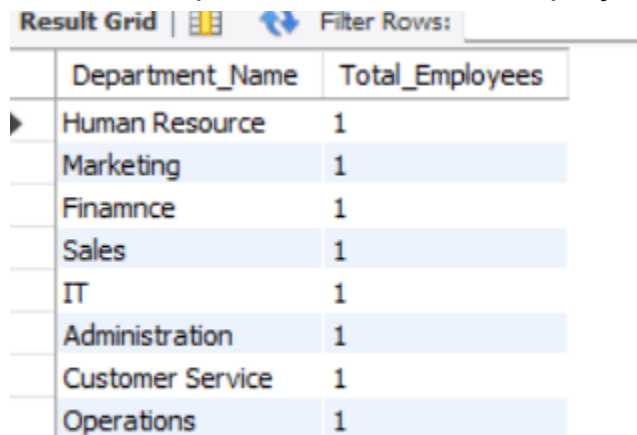
FROM department AS d

LEFT JOIN employees AS e

ON d.Department_id=e.Department_id

GROUP BY d.Department_Name;

To show department with null employees, Left join is used.



The screenshot shows a 'Result Grid' with a 'Filter Rows' button. The table has two columns: 'Department_Name' and 'Total_Employees'. It lists eight departments, each with a value of 1 in the 'Total_Employees' column.

Department_Name	Total_Employees
Human Resource	1
Marketing	1
Finamnce	1
Sales	1
IT	1
Administration	1
Customer Service	1
Operations	1

3.Right Join:

List all locations with names of employees assigned to each location with display of NULL for employee name if no employees are assigned.

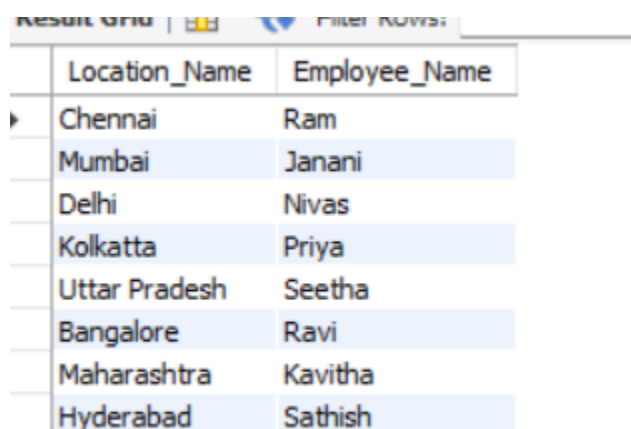
Command Used: SELECT l.Location_Name,e.Employee_Name

FROM employees AS e

RIGHT JOIN location AS l

ON l.Location_id=e.Location_id;

To display employee with no location, Right Join is used.



The screenshot shows a 'Result Grid' with a 'Filter Rows' button. The table has two columns: 'Location_Name' and 'Employee_Name'. It lists eight locations, each with an employee name. The first location, Chennai, has the employee 'Ram'.

Location_Name	Employee_Name
Chennai	Ram
Mumbai	Janani
Delhi	Nivas
Kolkatta	Priya
Uttar Pradesh	Seetha
Bangalore	Ravi
Maharashtra	Kavitha
Hyderabad	Sathish

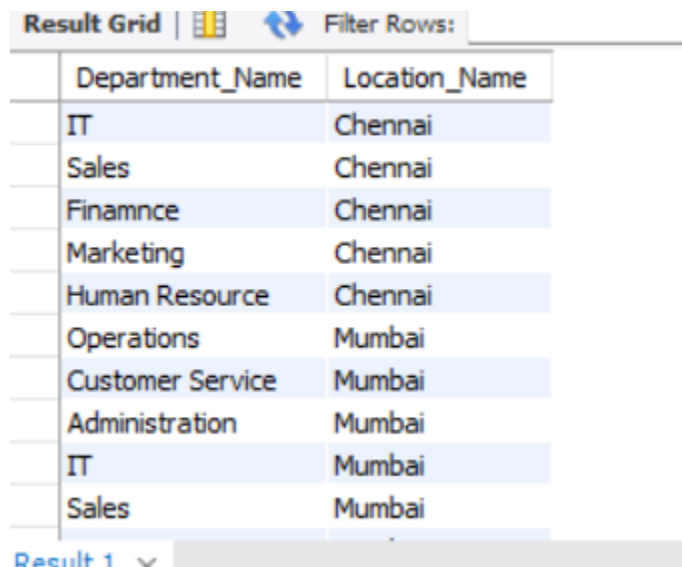
4. Cross Join:

Show all Possible combination of department and locations:

Command Used: SELECT d.Department_Name,l.Location_Name

FROM department AS d

CROSS JOIN location AS l;



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. It displays a table with two columns: 'Department_Name' and 'Location_Name'. The table contains 16 rows, representing all possible combinations of 8 departments and 2 locations. The departments listed are IT, Sales, Finance, Marketing, Human Resource, Operations, Customer Service, and Administration. The locations listed are Chennai and Mumbai. The first 8 rows show departments paired with Chennai, and the next 8 rows show the same departments paired with Mumbai.

Department_Name	Location_Name
IT	Chennai
Sales	Chennai
Finance	Chennai
Marketing	Chennai
Human Resource	Chennai
Operations	Mumbai
Customer Service	Mumbai
Administration	Mumbai
IT	Mumbai
Sales	Mumbai

In total, there are 8 different departments with 2 different locations. So there are 16 possible combinations.

5. Self Join:

Show pairs of employees working in same department:

Command Used: SELECT

e1.Employee_Name AS Employee1,

e2.Employee_Name AS Employee2,

e1.Department_id

FROM employees AS e1

INNER JOIN employees AS e2
 ON e1.Department_id=e2.Department_id
 AND e1.Employee_id<e2.Employee_id;

Result Grid		Filter Rows:		Export
	Employee1	Employee2	Department_id	

Two employees name are displayed with respective departments.

WINDOWS FUNCTION:

1.Window function query to rank employees by salary using rank()

Command Used:SELECT

Employee_Name,Salary,

RANK()OVER (ORDER BY Salary DESC) AS Salary_Rank

FROM employees;

Result Grid		Filter Rows:		Export:
	Employee_Name	Salary	Salary_Rank	
▶	Janani	62000.00	1	
	Ram	55000.00	2	
	Seetha	43000.00	3	
	Priya	35000.00	4	
	Kavitha	31000.00	5	
	Ravi	24000.00	6	
	Sathish	17000.00	7	
	Nivas	15000.00	8	

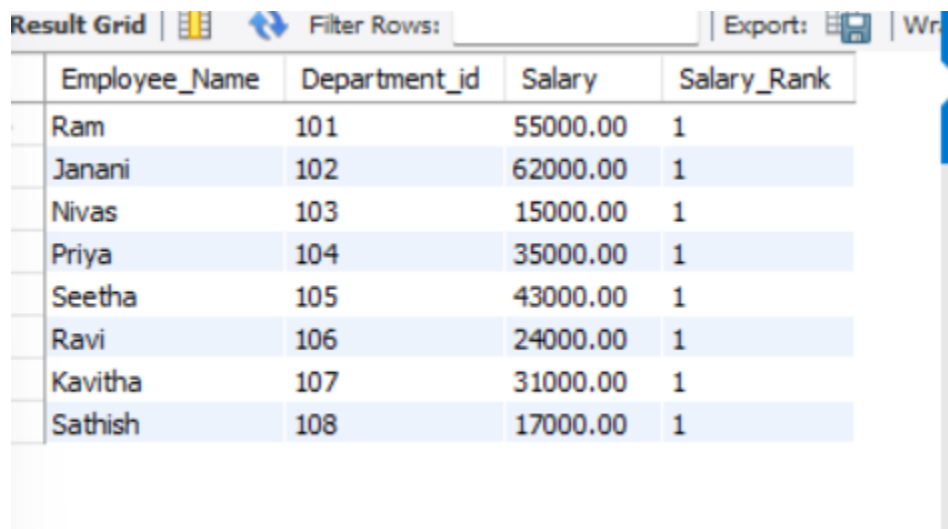
2.Window Function query to rank employees by salary within each department() using Dense Rank()

Command Used:SELECT

Employee_Name,Department_id,Salary,

Dense_Rank() OVER(PARTITION BY Department_id ORDER BY Salary
DESC)AS Salary_Rank

FROM employees;



The screenshot shows a 'Result Grid' window with a toolbar at the top containing icons for 'Filter Rows', 'Export', and 'Write'. The grid displays the following data:

Employee_Name	Department_id	Salary	Salary_Rank
Ram	101	55000.00	1
Janani	102	62000.00	1
Nivas	103	15000.00	1
Priya	104	35000.00	1
Seetha	105	43000.00	1
Ravi	106	24000.00	1
Kavitha	107	31000.00	1
Sathish	108	17000.00	1

3.Window function query for running total salary by department:

Command Used:SELECT

Employee_Name,Department_id,Salary,

SUM(Salary) OVER(PARTITION BY Department_id ORDER BY
Employee_id) AS Running_Total _Salary

FROM employees;

Result Grid

Filter Rows:

Export:

Wrap Cell Content

	Employee_Name	Department_id	Salary	Running_Total_Salary
▶	Ram	101	55000.00	55000.00
	Janani	102	62000.00	62000.00
	Nivas	103	15000.00	15000.00
	Priya	104	35000.00	35000.00
	Seetha	105	43000.00	43000.00
	Ravi	106	24000.00	24000.00
	Kavitha	107	31000.00	31000.00
	Sathish	108	17000.00	17000.00

Conclusion:

The employee database is created using clauses and operators such as order by, group by, joins and window functions are also used in this Project.

